Comparative Time Series Analysis and Forecasting of Mobile Network Traffic

Author: Nick-Lemy Kayiranga
Course: ML Techniques I - Formative Assignment
GitHub:
Video:

1. Introduction

This report presents a comprehensive analysis of mobile network traffic data recorded in Milan, Italy, released by Telecom Italia Mobile (TIM) as part of a public data challenge [1]. The dataset covers a continuous two-month period from November 2013 to January 2014 and measures internet activity across a 100x100 regular grid of 10,000 geographical areas at 10-minute intervals [2]. The grid geometry is described in a companion dataset [3].

Three tasks are addressed: (1) efficient loading and processing of approximately 20 GB of raw data within available memory constraints, (2) exploratory analysis of the spatial and temporal properties of the traffic signal, and (3) implementation and comparison of three forecasting models - one classical statistical approach (SARIMA) and two neural network approaches (LSTM and TCN) - for one-step-ahead prediction at three selected grid areas during the week of December 16-22, 2013.

Table Of Contents

2. Data Handling and Memory Management

2.1 Hardware and Software Setup

All experiments were run on my personal machine with an 8-core CPU, 16 GB RAM, and 245 GB free disk space running Arch Linux. Libraries used: Python 3.14.4, pandas 3.0.3, NumPy 2.4.6, PyTorch 2.12.0 (CPU), statsmodels 0.14.6, scikit-learn 1.8.0, and pyarrow 24.0.0. No GPU was available, which was the main constraint for neural network training time. With only 9.1 GB of free RAM and 20.5 GB of raw data, loading the dataset naively was not possible and shaped every design decision in the pipeline.

2.2 Loading Strategy and Memory Optimisations

The dataset arrives as 7 zip archives totalling 5.28 GB compressed, containing 62 daily tab-separated text files with a combined uncompressed size of 20.5 GB. Each file has 8 columns; only three are relevant: *square_id* (column 0), *time interval in milliseconds* (column 1), and *internet_activity* (column 7). Four optimisations were applied in sequence.

First, direct zip streaming: Python's zipfile module decompresses each file on the fly without writing anything to disk, eliminating the need for 20 GB of temporary disk space. Second, column selection using usecols=[0,1,7], loading only 3 of 8 columns and reducing per-file memory by approximately 62%. Third, group-sum aggregation: each raw file contains multiple rows per (square_id, time_interval) pair because different country codes are

recorded separately. A groupby().sum() per file collapses these into one row per area-timeslot pair. Fourth, float32 downcast: internet_activity is stored as float32 instead of float64, halving its memory footprint with negligible loss of precision for CDR-unit values.

The result is saved as a Snappy-compressed Parquet file (482 MB). All subsequent notebooks load this file directly, skipping the entire loading pipeline on re-runs.

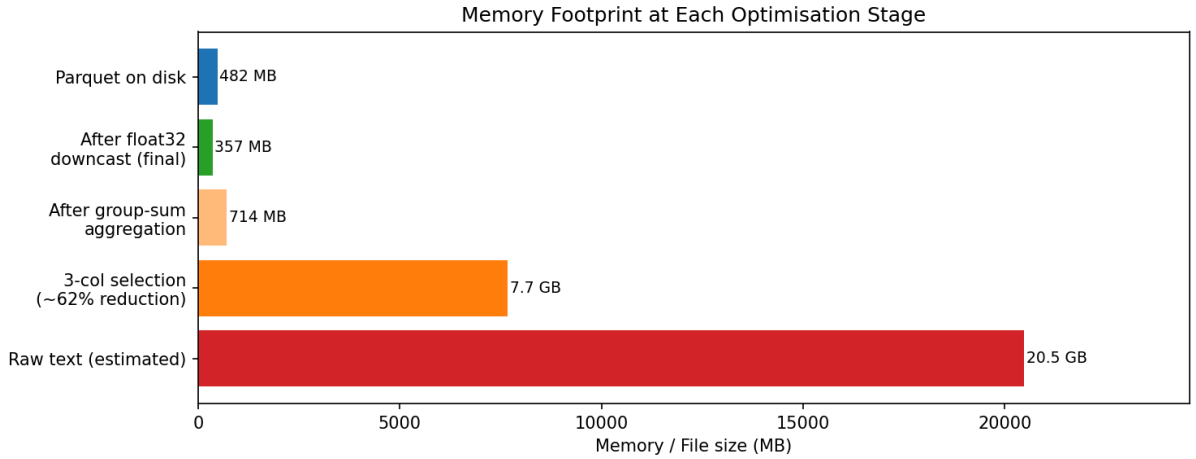

Figure 1 - Memory footprint at each optimisation stage.
Raw data: 20.5 GB; after 3-column
selection: ~7.7 GB; final float32 pivot: 357 MB; Parquet on disk: 482 MB.

2.3 Results

The final pivot table has shape 8,928 x 10,000: 8,928 ten-minute slots across 62 days and 10,000 grid areas. Process memory peaked at 553 MB during concatenation and settled at 357 MB for the float32 pivot, a 98.3% reduction from the raw 20.5 GB. Missing values (1.57% of cells, representing areas with zero internet activity in a given slot) were filled with 0.

2.4 Challenges

Five main challenges were encountered and resolved. Extraction was not feasible due to disk space constraints and was replaced by streaming from zip. Multiple rows per (area, time) from country-code splitting were resolved by two-pass groupby aggregation. Peak RAM during concatenation was managed by processing one daily file at a time before concatenating. Missing time slots were correctly interpreted as zero activity rather than data errors. Finally, raw Unix millisecond timestamps required conversion to a proper DatetimeIndex for time-series operations.

3. Exploratory Data Analysis

3.1 Traffic Distribution Across Areas

Figure 1: Traffic PDF Across Milan Grid Areas

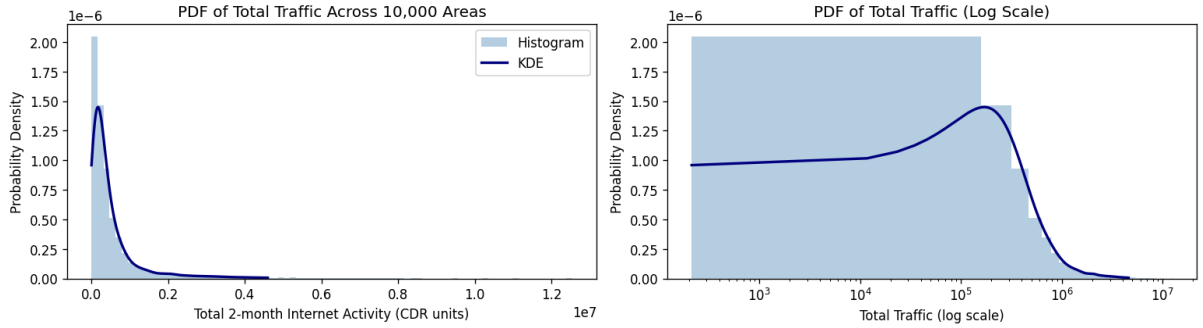

Figure 2 - PDF of total two-month internet activity across 10,000 grid areas (left: linear scale, right: log scale). The distribution is strongly right-skewed with a long tail of high-activity central areas.

The distribution of total two-month traffic across the 10,000 grid areas is strongly right-skewed. The median area generates approximately 273,000 CDR units over the two months, while the mean is 546,000 - nearly double - confirming that a small number of high-traffic areas pull the average up significantly. The maximum is 12,550,960 CDR units (Square 5161), more than 45 times the median. On the log scale the distribution approximates a log-normal shape, typical for spatial traffic data in cities. The bulk of grid squares correspond to low-activity suburban or industrial zones, while a small cluster of central, commercial, and transit areas concentrate a disproportionate share of mobile activity. This heterogeneity is consistent with Milan's urban structure where the historic centre, the central train station, and major commercial districts generate orders of magnitude more traffic than the outskirts.

3.2 Time Series of the Three Target Areas

Three areas are used throughout the analysis and forecasting tasks: Square 5161 (highest total traffic: 12,550,960 CDR), Square 4159 (2,399,680 CDR), and Square 4556 (4,502,657 CDR).

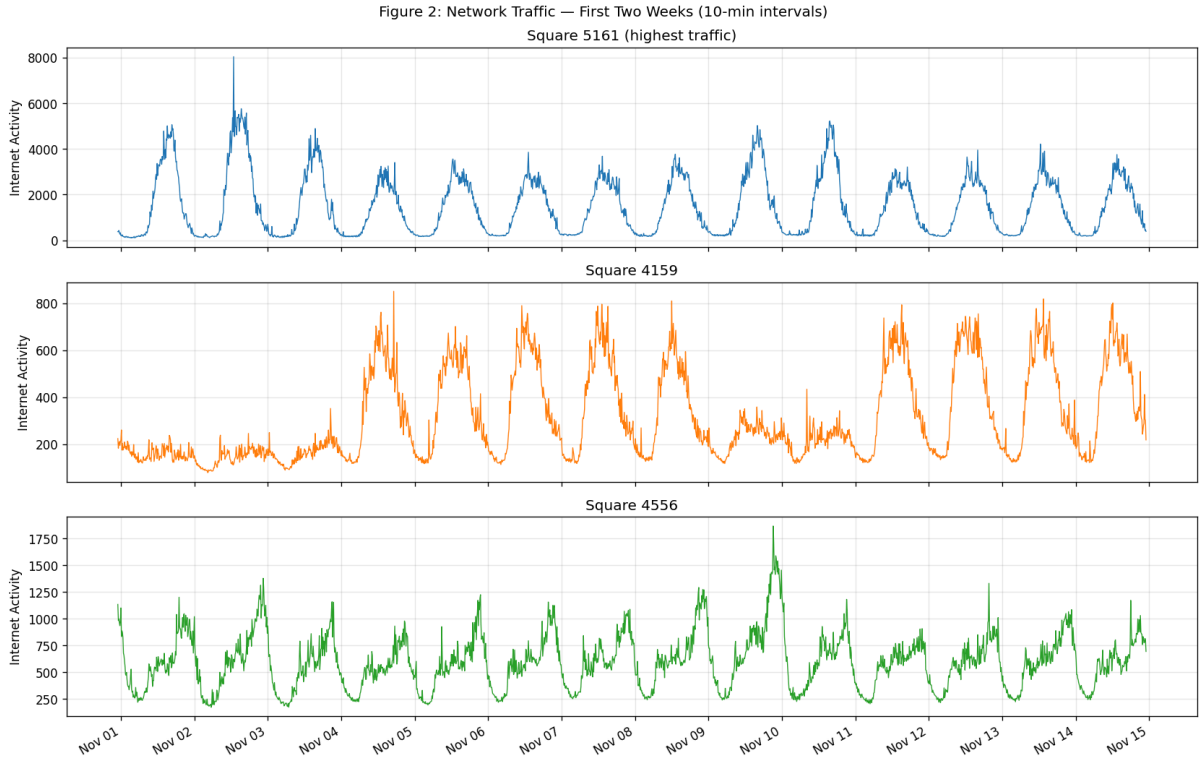

Figure 3 - Traffic time series for the three target areas over the first two weeks. Top: Square 5161. Middle: Square 4159. Bottom: Square 4556.

All three areas show a clear repeating daily cycle with peaks during daytime hours and sharp drops overnight, and a weekly structure with lower activity on weekends. Square 5161 has the largest and most pronounced peaks, suggesting a busy transit or commercial hub such as the Milan central station area. Square 4159 shows a more moderate and smoother pattern typical of a mixed residential-commercial zone. Square 4556 sits between the other two with sharp daytime spikes indicating an active commercial location.

3.3 Stationarity and Rolling Statistics

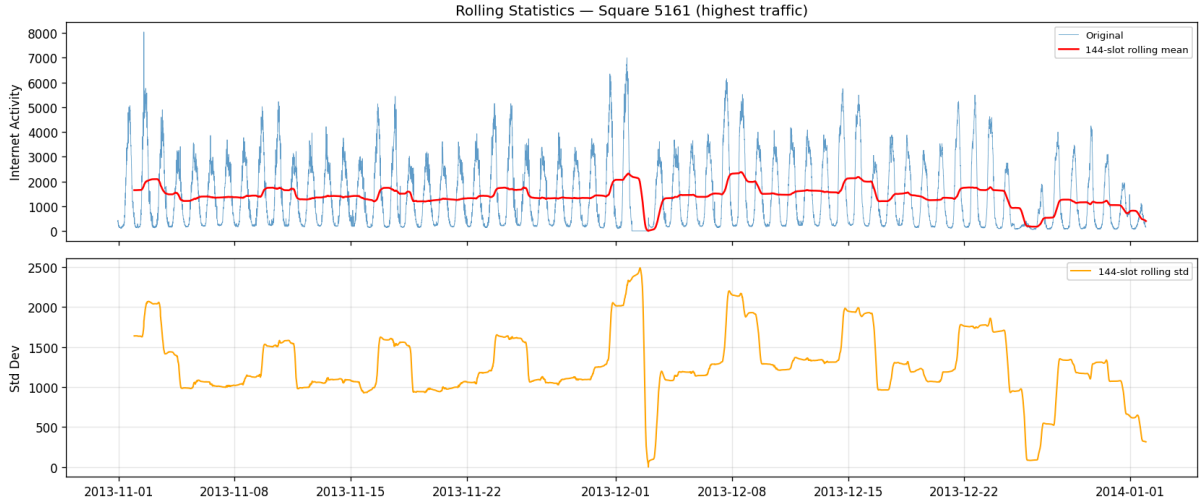

Figure 4 - Rolling mean and standard deviation for Square 5161 (window = 144 slots = 1 day). The stable rolling statistics confirm stationarity.

The rolling mean oscillates regularly around a stable level with no long-term trend. The rolling standard deviation stays consistent throughout the two months. The Augmented Dickey-Fuller (ADF) test formally confirms stationarity for all three areas. The ADF test evaluates the null hypothesis that a series has a unit root (non-stationary). Test statistics were -18.63 for Square 5161, -12.44 for Square 4159, and -11.71 for Square 4556, all well below the 1% critical value of -3.43. The p-values are effectively zero in all cases. Stationarity confirms that no differencing is needed before fitting SARIMA, which simplifies model specification

3.4 Time Series Decomposition

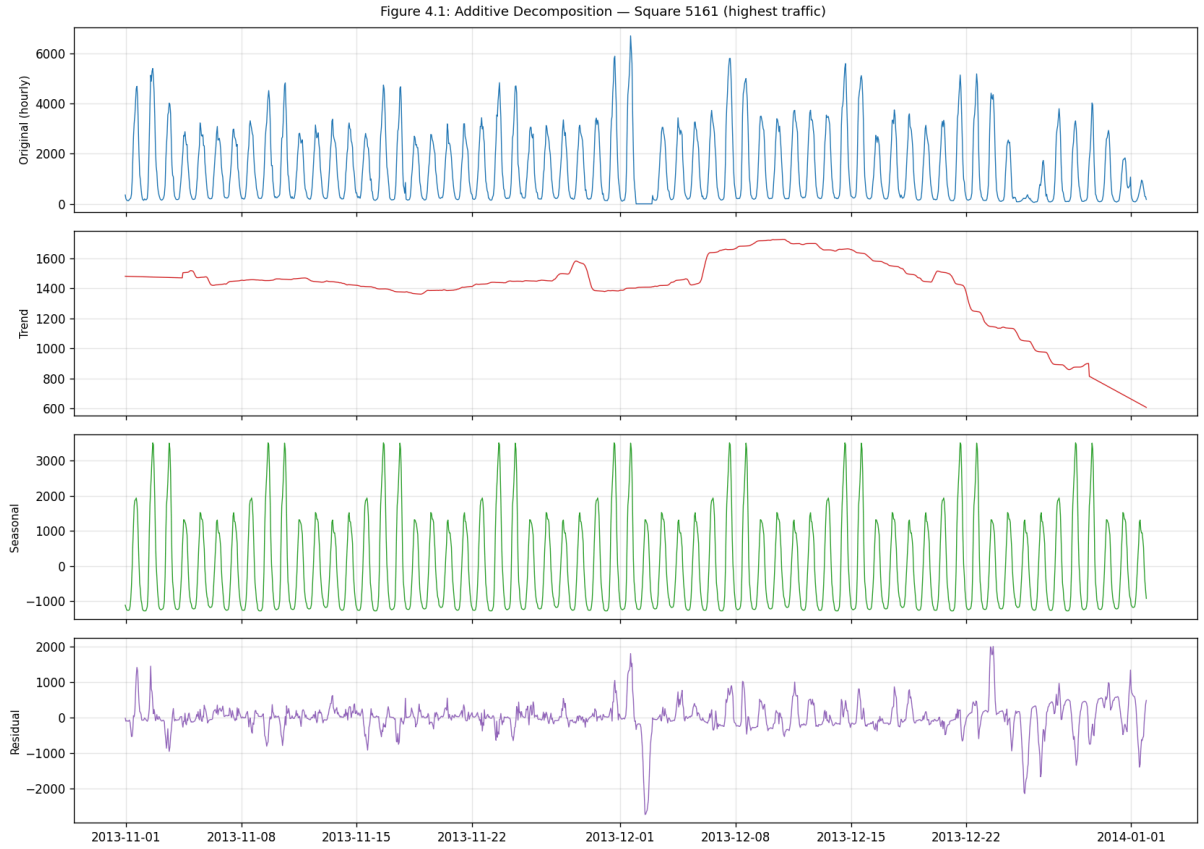

Figure 5 - Additive decomposition of Square 5161 (hourly resolution, weekly seasonal period). The seasonal component dominates; trend is nearly flat; residuals are small.

Additive decomposition at hourly resolution using a weekly seasonal period (168 hours) separates each series into trend, seasonal, and residual components. The seasonal component dominates, capturing both the daily sub-cycle (morning build-up, midday peak, overnight drop) and the weekly structure (lower weekends). The trend component shows no strong directional movement over the two months. The residuals are small relative to the seasonal amplitude, confirming that the seasonal model explains most of the variance. Occasional

residual spikes indicate isolated events that motivate a stochastic component in the forecasting models.

3.5 Autocorrelation and Partial Autocorrelation

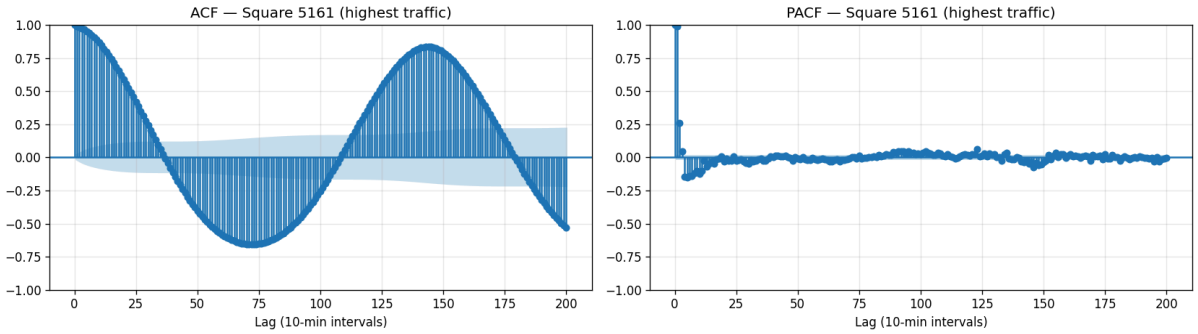

Figure 6 - ACF (left) and PACF (right) for Square 5161. Spike at lag 144 confirms the daily cycle; PACF cutoff after lag 2 suggests AR(2) structure.

The ACF decays slowly and shows clear spikes at multiples of lag 144 (one full day at 10-minute resolution), confirming strong daily periodicity. A secondary set of spikes near lag 1008 (one week) confirms the weekly seasonal component. The PACF shows significant partial autocorrelations at the first two lags then a sharper cutoff, suggesting AR(2) short-range dependency. These findings directly informed the SARIMA specification: non-seasonal order AR(2), MA(1), and a seasonal period of 24 at hourly resolution.

3.6 Spatial Analysis

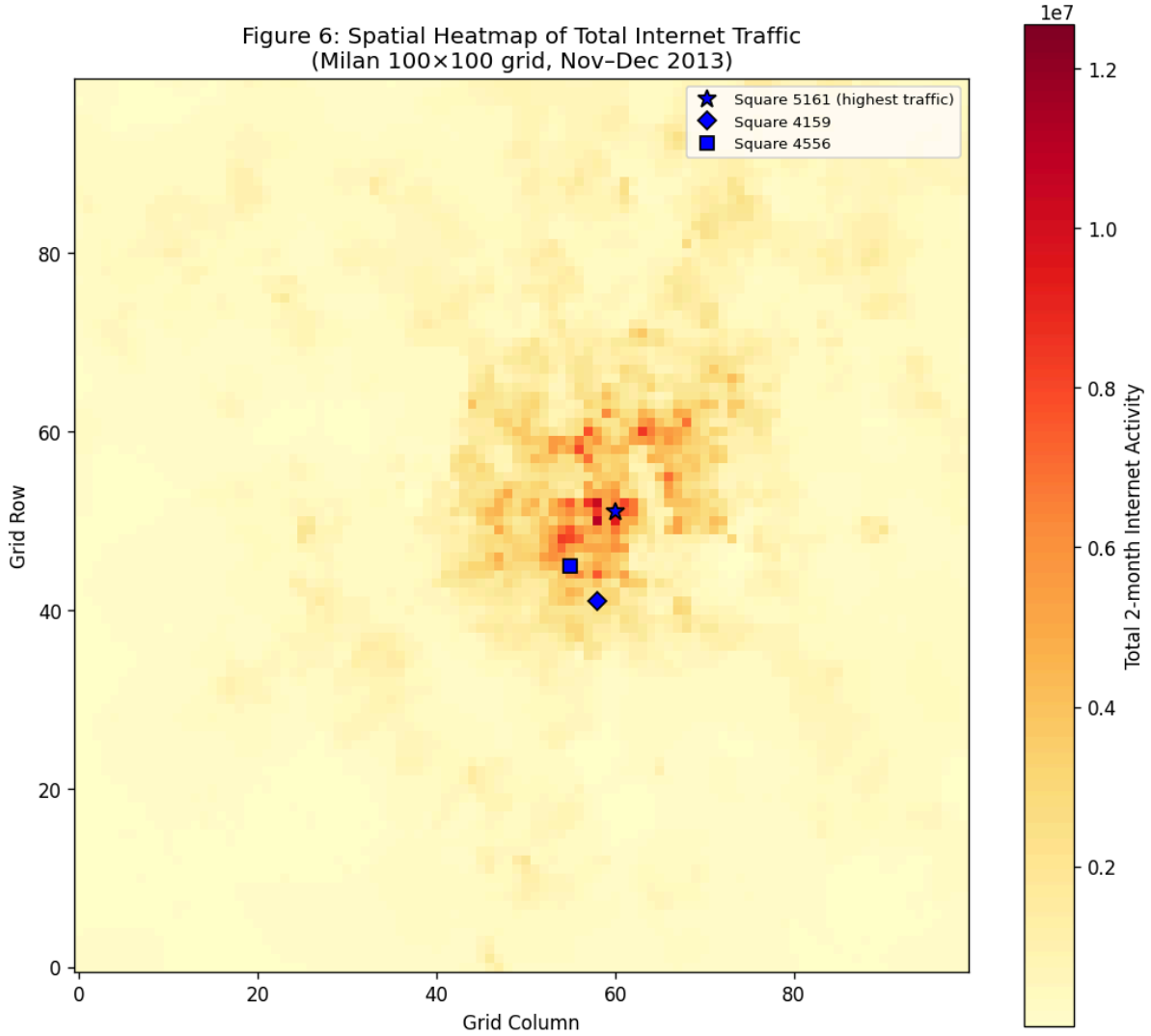

Figure 7 - Spatial heatmap of total two-month traffic across the 100x100 Milan grid. The three target areas are marked. Square 5161 is the peak (star marker).

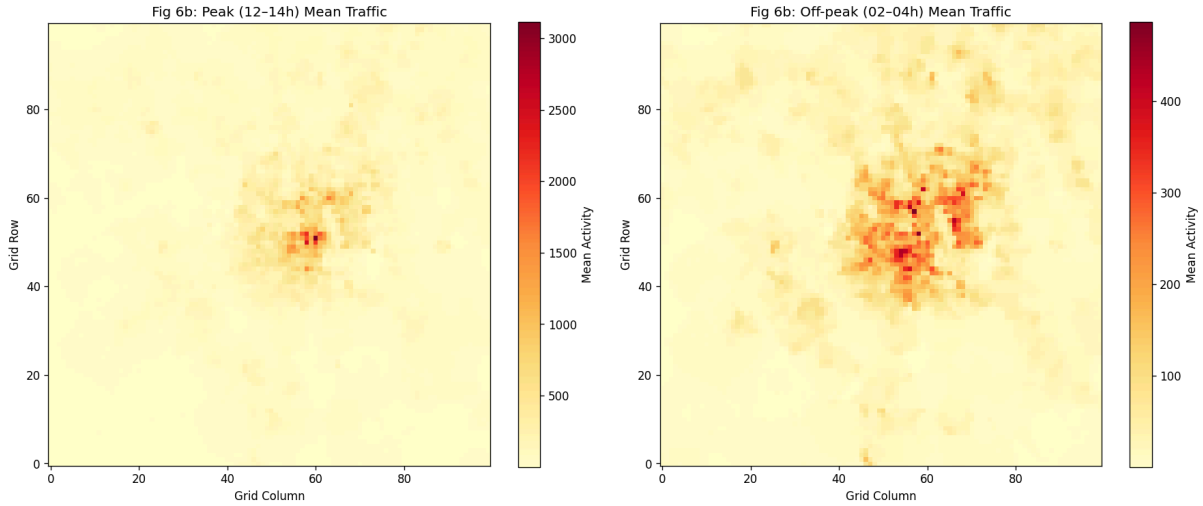

Figure 8 - Mean traffic during peak hours (12-14h, left) vs. off-peak (02-04h, right).

Traffic concentrates in a central corridor roughly spanning rows 45-60 and columns 50-65, corresponding to the city centre and its surrounding high-density districts. Square 5161 (row 51, column 61) stands out as the peak. Traffic drops off sharply toward the grid edges which cover suburban and semi-rural areas. The peak vs. off-peak comparison shows that central hotspots are much more pronounced during peak hours (12-14h), reflecting lunch-hour concentration in offices and commercial zones. During off-peak hours (02-04h), the relative spatial distribution is preserved at much lower absolute levels.

3.7 Anomalies and Unusual Patterns

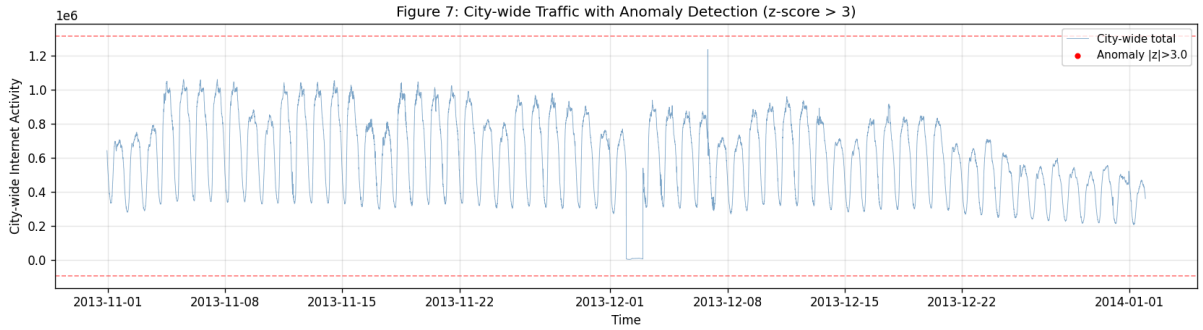

Figure 9 - City-wide traffic over the full two months. No z-score > 3 outliers found at aggregate level.

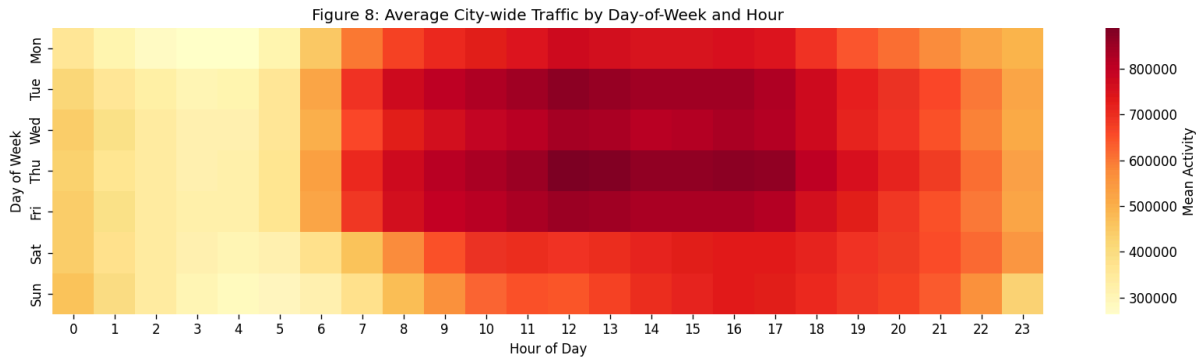

Figure 10 - Average traffic by day of week and hour of day.

No city-wide slots exceeded a z-score of 3 at aggregate level, expected because summing across 10,000 areas smooths individual spikes. The day-of-week and hour-of-day heatmap reveals clear structural patterns: weekdays show the highest activity concentrated between 09:00 and 18:00 with a pronounced lunch peak; Saturday shows shifted and slightly lower activity; Sunday is consistently the lowest day across all hours. Notable calendar effects include a visible dip around December 25 and elevated activity around December 31. These event-driven deviations from the weekly pattern represent a structural challenge for forecasting models trained on historical averages.

4. Forecasting Model Design and Implementation

All models were trained on November 1 to December 15, 2013 (6,486 ten-minute slots) and evaluated on December 16-22 (1,008 ten-minute slots). Each model is trained and evaluated independently per area. The test week is used exclusively for evaluation and is not part of training or validation in any form.

4.1 Model 1: SARIMA

SARIMA (Seasonal AutoRegressive Integrated Moving Average) [6] extends ARIMA with seasonal terms, written as SARIMA(p,d,q)(P,D,Q)[s] where p and q are the non-seasonal AR and MA orders, d is the differencing order, and P, Q, D, s are the corresponding seasonal parameters at period s.

Given that the ADF test confirmed stationarity (d=0), ACF/PACF analysis suggested AR(2) and MA(1), and the decomposition showed a strong daily cycle, the final specification is SARIMA(2,0,1)(1,1,1)[24]. The series is resampled to hourly resolution before fitting, reducing the seasonal period from 144 to 24 and making the model computationally tractable. Training uses Maximum Likelihood Estimation via statsmodels SARIMAX [8]. Outliers above the 99th percentile are clipped before fitting to improve convergence. Prediction is a batch 168-step forecast covering the 7-day test period. No normalisation is applied.

4.2 Model 2: LSTM

LSTM (Long Short-Term Memory) [4] is a recurrent neural network that learns long-range dependencies through gated memory cells (input gate, forget gate, output gate), allowing the network to selectively retain or discard information across many time steps.

Architecture: Input(144, 1) -> LSTM(64 units, 2 layers, dropout=0.2) -> Linear(1). Input representation: sliding windows of 144 consecutive 10-minute traffic values (one full day of history). All values are normalised to [0,1] using MinMaxScaler fitted on training data only. Training uses Adam optimiser (lr=1e-3), MSE loss, batch size 64, and early stopping with patience=7 on a 10% validation split held from the end of the training period. Predictions over the test week use auto-regressive rollout via PyTorch [7]: each predicted value is appended to the input buffer and the window slides forward.

4.3 Model 3: TCN

TCN (Temporal Convolutional Network) [5] applies stacked dilated causal convolutions to time series. Dilation factors grow exponentially (1, 2, 4) so each successive block sees a wider effective receptive field without increasing the parameter count. Unlike RNNs, TCNs process sequences in parallel during training.

Architecture: Input(144, 1) -> 3 dilated residual Conv1D blocks (64 channels, kernel size 3, dilation 1/2/4, BatchNorm, Dropout=0.2) -> final time step -> Linear(1). Same input

representation, normalisation, training procedure, and auto-regressive prediction strategy as LSTM.

5. Hyperparameter Tuning

Tuning was performed on Square 4159 as a representative area. For SARIMA, the order was selected by comparing AIC scores: SARIMA(1,0,1)(1,1,1)[24] gave AIC = 10,376 and SARIMA(2,0,1)(1,1,1)[24] gave AIC = 10,362, which was chosen as the final specification.

5.1 LSTM Experiments

The key finding is that a 1-day (144-slot) window captures the full daily cycle and outperforms both shorter (72 slots) and longer (288 slots) windows. Increasing hidden units from 64 to 128 added training time without improving accuracy, and reducing to 1 layer performed worse than 2 layers.

5.2 TCN Experiments

Increasing channels from 32 to 64 improved performance noticeably. Adding a fourth dilated block did not consistently help, suggesting 3 blocks is sufficient for a 144-step input sequence. A larger kernel (size 5) and a longer window (288 slots) with lower learning rate did not outperform the baseline 64-channel 3-block configuration.

6. Results

6.1 Prediction Plots

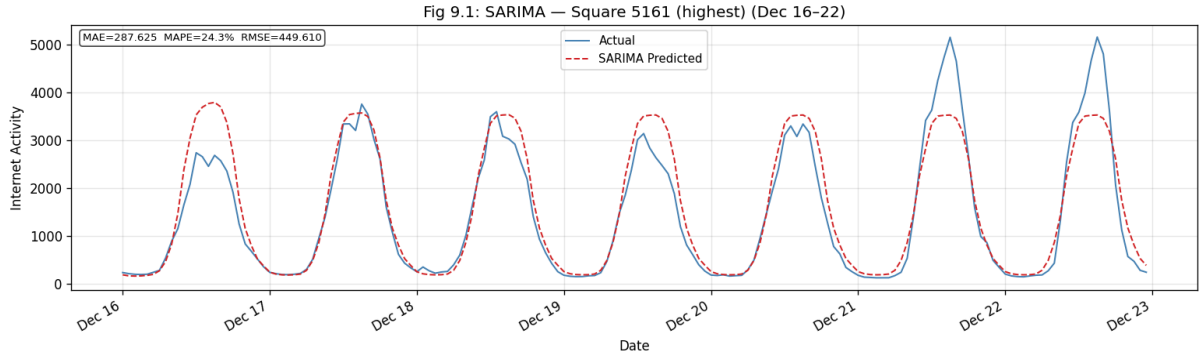

Figure 11 - SARIMA predictions vs. actual traffic, Square 5161, December 16-22.

5.1 / 5.2 Hyperparameter Tuning · Detailed Results

Table 5 · LSTM experiments (Square 4159)

Best: LSTM-D MAE=146.17 MAPE=79.7% RMSE=182.78 ? Shallower (1 layer): test if depth helps or hurts.

Table 6 · TCN experiments (Square 4159)

Best: TCN-E MAE=102.4 MAPE=44.0% RMSE=127.57 ? Longer window (2 days) with lower LR.

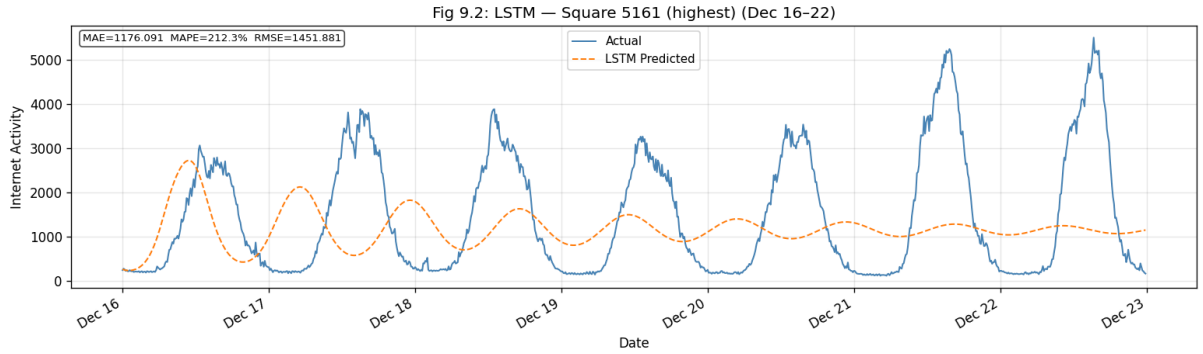

Figure 12 - LSTM predictions vs. actual traffic, Square 5161, December 16-22.

6.2 Performance Tables

MAE (Mean Absolute Error) measures the average absolute prediction error in CDR units and gives a direct, interpretable measure of how far predictions are from reality. MAPE (Mean Absolute Percentage Error) is scale-independent, making it useful for comparing performance across the three areas which have very different traffic magnitudes. RMSE (Root Mean Squared Error) penalises large individual errors more heavily than MAE and is sensitive to occasional big misses.

Table 1 - Square 5161 (highest traffic)

Model	MAE	MAPE (%)	RMSE
SARIMA	287.62	24.32	449.61
LSTM	1176.09	212.25	1451.88
TCN	1105.43	145.02	1488.08

Table 2 - Square 4159:

Model	MAE	MAPE (%)	RMSE
SARIMA	66.42	37.75	111.15
LSTM	229.59	139.86	266.59
TCN	103.22	37.36	139.69

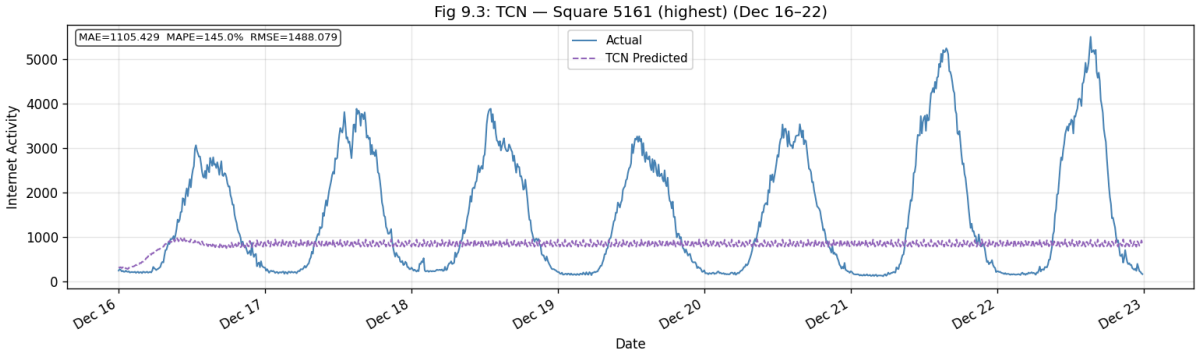

Figure 13 ? TCN predictions vs. actual traffic, Square 5161, December 16?22, 2013.

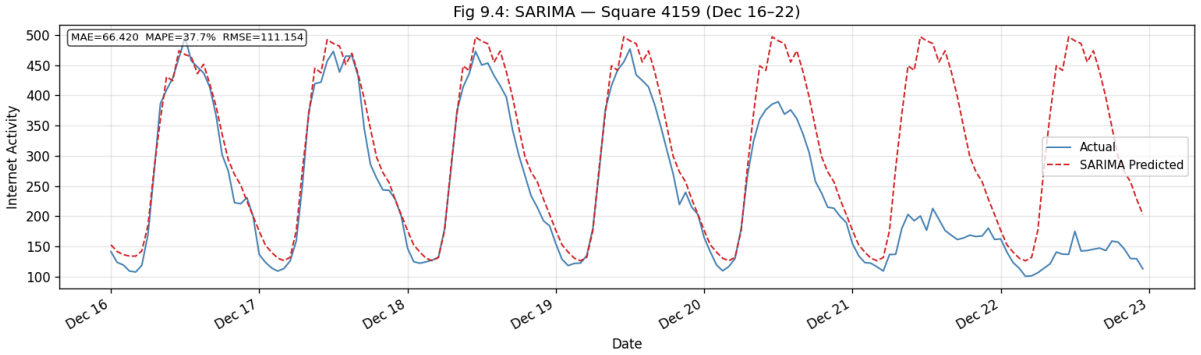

Figure 14 ? SARIMA predictions vs. actual traffic, Square 4159, December 16?22, 2013.

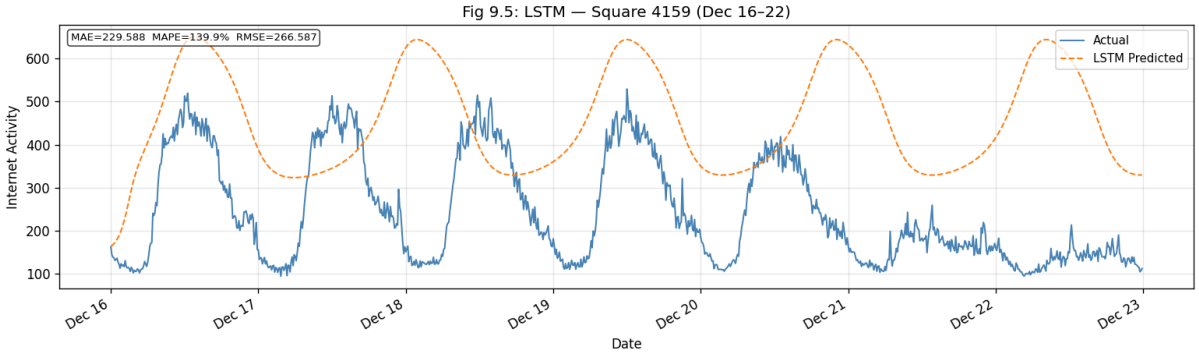

Figure 15 ? LSTM predictions vs. actual traffic, Square 4159, December 16?22, 2013.

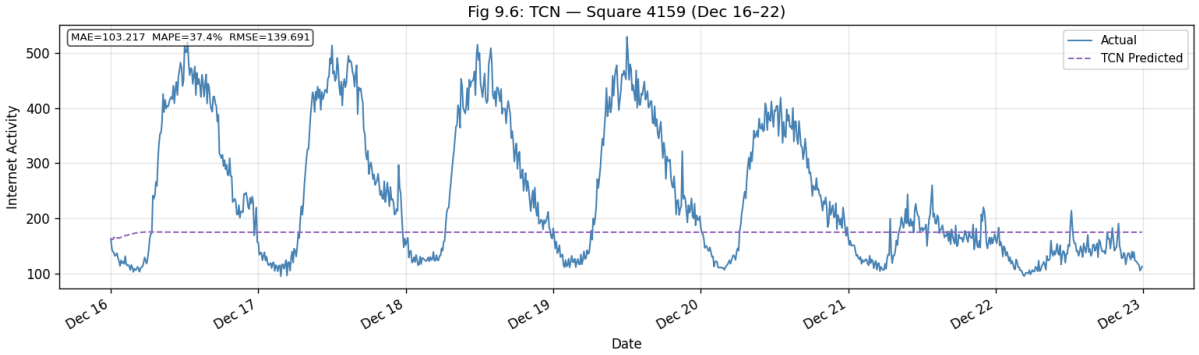

Figure 16 ? TCN predictions vs. actual traffic, Square 4159, December 16?22, 2013.

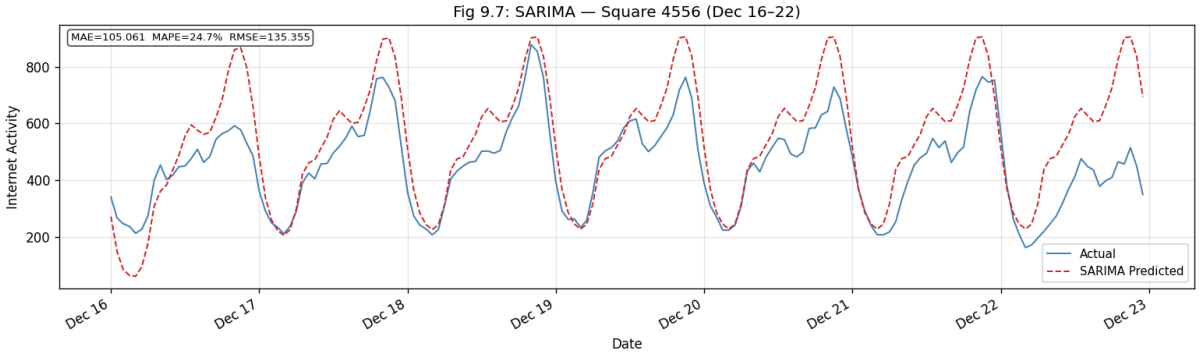

Figure 17 ? SARIMA predictions vs. actual traffic, Square 4556, December 16?22, 2013.

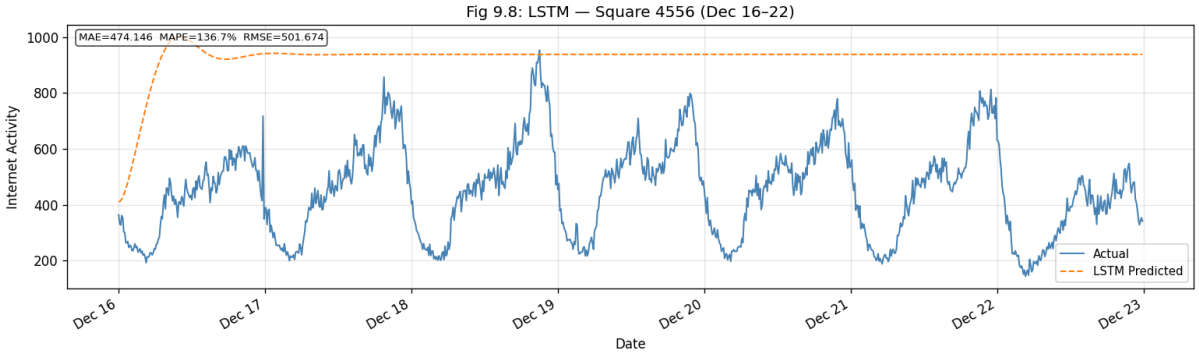

Figure 18 ? LSTM predictions vs. actual traffic, Square 4556, December 16?22, 2013.

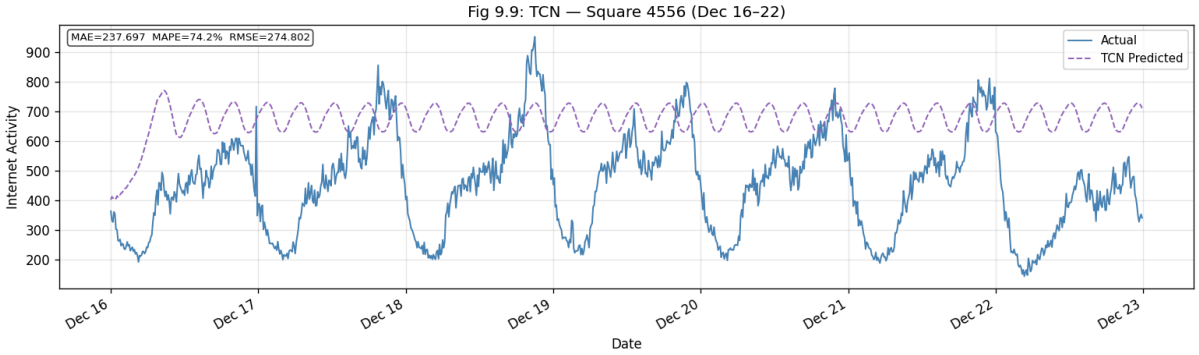

Figure 19 ? TCN predictions vs. actual traffic, Square 4556, December 16?22, 2013.

Table 3 - Square 4556:

Model	MAE	MAPE (%)	RMSE
SARIMA	105.06	24.66	135.36
LSTM	474.15	136.70	501.67
TCN	237.70	74.22	274.80

6.3 Training and Execution Time

Times are averaged across the three areas and measured on CPU (8-core, no GPU). Training time is the wall-clock time to fit the model on one area's full training data. Execution time is the time to generate predictions for the entire 7-day test period.

Table 4 - Timing:

Model	Avg Train Time (s)	Avg Exec Time (s)
SARIMA	29.2	0.020
LSTM	162.5	1.600
TCN	290.4	2.052

7. Comparative Analysis and Discussion

SARIMA is the best-performing model across all three areas and all three metrics. For Square 5161 it achieves MAE = 287.6 and MAPE = 24.3%, compared to LSTM at MAPE = 212.3% and TCN at MAPE = 145.0%. For Square 4159, SARIMA reaches MAE = 66.4 and MAPE = 37.8%, with TCN close behind at MAPE = 37.4% but LSTM far worse at 139.9%. For Square 4556, SARIMA leads with MAE = 105.1 and MAPE = 24.7%. SARIMA is also the fastest to train at 29 seconds on average, versus 163 seconds for LSTM and 290 seconds for TCN.

The high MAPE values for LSTM and TCN stem from the auto-regressive prediction strategy over the 7-day test window. Each predicted value is fed back as input for the next step, so small early errors compound across 1,008 ten-minute intervals. SARIMA avoids this because it explicitly encodes the seasonal structure and does not rely purely on its own past predictions. The EDA findings from Section 3 support this: all series are stationary with a

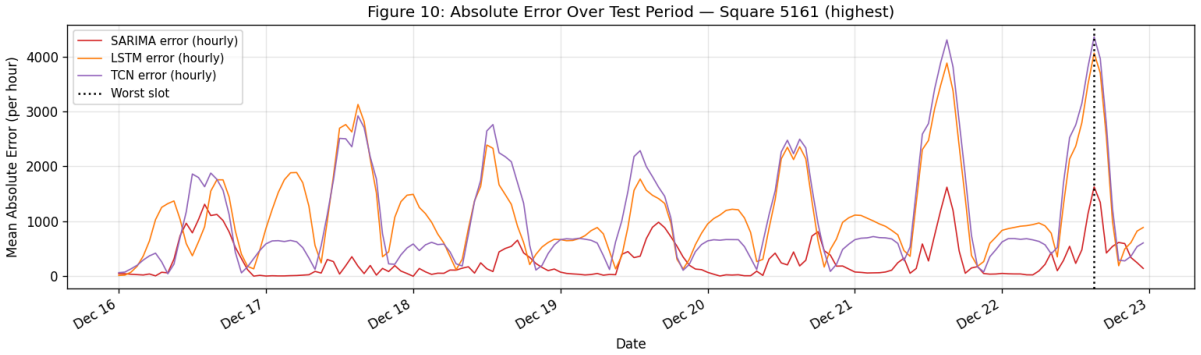

Figure 20 ? Absolute prediction error over December 16?22 for all three models (Square 5161, resampled to hourly). Worst slot (December 22, 15:00 UTC) marked with dotted vertical line.

dominant, stable weekly-daily seasonal pattern and no long-term trend, which are exactly the conditions where SARIMA performs best.

TCN outperforms LSTM on two out of three areas (Square 4159 and 4556). TCNs benefit from parallel processing during training and their dilated convolutions provide a large effective receptive field without the vanishing gradient problem that can affect deep LSTMs. However TCN takes the longest to train (290 seconds on average).

Possible improvements for the neural networks include: rolling one-step-ahead prediction where the model is updated with each true observation rather than feeding back its own prediction; adding time-of-day and day-of-week encodings as explicit input features; incorporating data from neighbouring grid squares as spatial context.

8. Failure Analysis

Figure 20 - Absolute prediction error over the test week for all three models on Square 5161, resampled to hourly resolution. The worst slot (December 22, 15:00 UTC) is marked with a dotted vertical line.

The worst-performing slot across all models is December 22, 2013 at 15:00 UTC (16:00 local Milan time), with a combined mean absolute error of 3,349 CDR units. Individual errors at that slot are: SARIMA 1,627, LSTM 4,062, TCN 4,358.

December 22 is a Sunday falling three days before Christmas. This combination produces an unusual traffic pattern: higher-than-usual afternoon retail and pre-holiday travel activity pushes traffic well above the regular Sunday afternoon profile that all models learned from training data. All models predict a typical Sunday seasonal pattern and therefore underestimate the spike.

SARIMA handles this slightly better because its explicit seasonal structure gives a more calibrated baseline prediction for that time of day and day of week. The LSTM and TCN have already accumulated prediction errors over six days of auto-regressive forecasting by the time they reach December 22, making them more sensitive to the spike.

This failure case illustrates a general limitation shared by all three approaches: they cannot anticipate calendar anomalies without external features such as a holiday indicator or event calendar as additional input. Future work could incorporate binary holiday flags or a calendar-aware embedding to address this.

9. Conclusion

This project analysed two months of Milan mobile internet traffic across 10,000 geographical areas and compared three forecasting models for one-step-ahead prediction during December 16-22, 2013.

The main data handling contribution was a memory-efficient pipeline that streams directly from zip archives, selects only the three relevant columns, aggregates by area and time slot, and stores the result as a float32 Parquet file, reducing working memory from 20.5 GB to 357 MB (98.3% reduction).

The exploratory analysis revealed a strongly right-skewed spatial traffic distribution, a dominant weekly-daily seasonal pattern, confirmed stationarity for all three target areas, and clear spatial concentration of activity in the Milan city centre. These findings directly motivated the SARIMA model specification.

SARIMA was the best-performing model with MAPE values of 24-38% versus 37-212% for the neural networks. Its advantage stems from explicitly encoding the seasonal structure of the data data, avoiding the error accumulation that affects auto-regressive LSTM and TCN approaches over a 7-day horizon. SARIMA is also the fastest to train. The identified failure case - a pre-Christmas Sunday afternoon spike on December 22 - highlights the general limitation of models that cannot account for calendar anomalies without external features.

References

[1] G. Barlacchi et al., "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," Sci Data 2, 150055 (2015). https://doi.org/10.1038/sdata.2015.55

[2] TIM Milan Telecommunications Dataset. Harvard Dataverse. https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/EGZHFV

[3] TIM Milan Grid Dataset. Harvard Dataverse. https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/QJWLFU

[4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, 9(8):1735-1780, 1997.

[5] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," arXiv:1803.01271, 2018.

[6] G. E. P. Box et al., "Time Series Analysis: Forecasting and Control," 5th ed., Wiley, 2015.

[7] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS 2019.

[8] S. Seabold and J. Perktold, "statsmodels: Econometric and Statistical Modeling with Python," 9th Python in Science Conf., 2010.

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," JMLR 12, pp. 2825-2830, 2011.

[10] The pandas development team, "pandas-dev/pandas," Zenodo, 2020. https://doi.org/10.5281/zenodo.3509134